BÖLÜM 6 / MODULE 6

Eşzamanlı Koleksiyonlar & Thread-Safe Veri Yapıları

Concurrent Collections & Thread-Safe Data Structures

ConcurrentHashMap iç mekaniği (CAS ve Kova Kilitleme), CopyOnWriteArrayList anlık görüntü kopyalaması, BlockingQueue varyantları, SynchronousQueue sıfır kapasite takası ve Lock-Free kuyruklar.

ConcurrentHashMap internals (CAS and Bin-Level locking), CopyOnWriteArrayList snapshot iteration, BlockingQueue variants, SynchronousQueue zero-capacity handoffs, and lock-free queues.

31

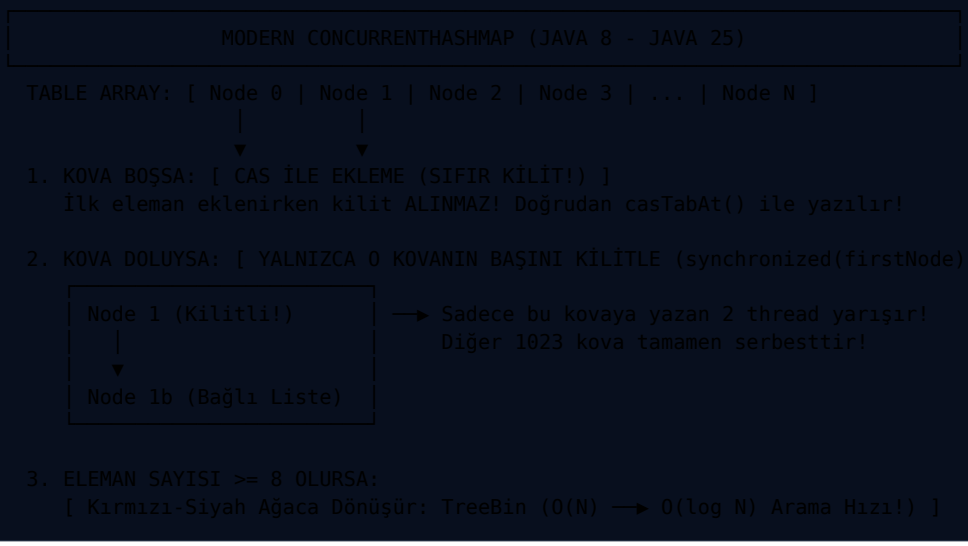
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

ConcurrentHashMap İç Mimarisi: Segment Yok! Node CAS, Bin Kilitleri & TreeBin

ConcurrentHashMap Internals: Node CAS, Bin-Level Locking & TreeBins

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski usul senkronize haritalar (Hashtable), 1000 çekmeceli bir kütüphanenin kapısına tek bir kilit takmak gibidir. Biri 1. çekmeceden kitap alırken diğer 999 kişi kapıda bekler. ConcurrentHashMap ise her çekmecenin başına küçük bir şifreli kilit takar! Biri 1. çekmeceyi açarken, diğeri 500. çekmeceden rahatça kitabını alır; birbirlerini hiç görmezler. Eğer bir çekmecede çok fazla dosya birikirse, çekmece otomatik olarak açılıp alfabetik bir fihrist ağacına (TreeBin) dönüşür!

🇬🇧 ENGLISH (ELI5)

A synchronized map (`Hashtable`) locks the entire bank vault whenever someone accesses safety deposit box 1 ConcurrentHashMap locks ONLY safety deposit box 1 (`synchronized(firstNode)`). Customers accessing boxes 2 through 1024 proceed concurrently with ZERO friction! Even better: if a box is empty, it inserts the item using atomic lock-free CAS without locking at all! If a single bucket accumulates more than 8 colliding items, it dynamically transmutes into a Red-Black Tree (`TreeBin`), preserving O(log N) search speed.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 8+ ve Java 25 ConcurrentHashMap İnovasyonları: 1) Segmentlerin Kaldırılması: Java 7'deki 16'lı `Segment[]` mimarisi çöpe atıldı. Artık doğrudan `Node\<K,V>[] table` dizisi kullanılır. 2) CAS ile İlk Ekleme: Kova (Bucket) boşsa, `Unsafe.compareAndSetObject` (modern JDK'da VarHandle) kullanılarak kilit almadan tek çevrimde yazılır. 3) Kova Düzeyinde Kilitleme: Kova doluysa sadece o kovanın ilk elemanı (`synchronized(node)`) kilitlenir; haritanın geneli açık kalır. 4) Treeify Eşiği: Tek bir kovadaki eleman sayısı 8'i geçerse ve tablo boyutu >= 64 ise bağlı liste kırmızı-siyah ağaca (`TreeBin`) dönüştürülür. Eleman sayısı 6'ya düşerse tekrar listeye (Untreeify) döner. 5) size() Hesabı: LongAdder mantığında `CounterCell[]` kullanılarak kilit almadan hesaplanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot ConcurrentHashMap Topology: 1) Demise of Segmented Locking: Java 8 discarded the historical 16-Segment structure. The map operates directly on a volatile `Node\<K,V>[] table`. 2) CAS-Driven Empty Bucket Ingestion: If `tabAt(i) == null`, the entry is inserted via `casTabAt()` with zero lock acquisition. 3) Bin-Level Synchronized Locks: When collisions occur, the thread synchronizes strictly on the root `Node` of that specific hash bin. Concurrent threads hashing to distinct bins execute in parallel. 4) TreeBin Transformation: If bin collisions reach `TREEIFY_THRESHOLD = 8` and table capacity >= 64, the singly-linked list transmutes into a balanced Red-Black Tree (`TreeNode`), bounding worst-case lookup from $O(N)$ to O(log N). 5) High-Concurrency Size Aggregation: `mappingCount()` leverages internal `CounterCell` arrays, mirroring `LongAdder` to prevent CAS contention.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ConcurrentHashMap ile Atomik computeIfAbsent ve İstatistikler
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.LongAdder;

public class HighThroughputUrlTracker {
    // URL bazlı istek sayaçları
    private final ConcurrentHashMap<String, LongAdder> accessCounters = new ConcurrentHashMap<>();

    public void trackVisit(String url) {
        // ⚠ ATOMİK: URL yoksa LongAdder oluştur, varsa eskisini getir (KİLİTSİZ!)
        accessCounters.computeIfAbsent(url, _ -> new LongAdder()).increment();
    }

    public long getVisitCount(String url) {
        var adder = accessCounters.get(url);
        return adder != null ? adder.sum() : 0L;
    }

    void main() {
        var tracker = new HighThroughputUrlTracker();
        tracker.trackVisit("https://maang.com/careers");
        tracker.trackVisit("https://maang.com/careers");
        IO.println("Sayfa Ziyaret Sayısı: " + tracker.getVisitCount("https://maang.com/careers"));
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı CHM Tuzağı: 'ConcurrentHashMap içine `null` key veya `null` value koyabilir miyiz?' Yanıt: 'KESİNLİKLE HAYIR! HashMap null kabul eder ama ConcurrentHashMap null key veya null value'ya ASLA izin vermez (NPE fırlatır). Çünkü çoklu thread ortamında map.get(k) == null döndüğünde 'Anahtar gerçekten yok mu, yoksa değeri mi null?' belirsizliği (ambiguity) kilit-siz çözülemez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory Interview Axiom: 'Can ConcurrentHashMap store null keys or null values?' Senior Answer: 'NEVER! `HashMap` permits nulls, but `ConcurrentHashMap` explicitly forbids null keys and null values, throwing `NullPointerException`. Doug Lea designed this because in a multithreaded map, `map.get(key) == null` cannot disambiguate whether the key is missing or the value is legitimately null without holding a lock across `containsKey()`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentHashMap boş kovalara kilit almadan CAS ile eleman ekler.
- Kova dolduğunda tüm tablo değil, sadece o kovanın ilk elemanı kilitlenir.
- ConcurrentHashMap null key ve null value kabul etmez (NullPointerException).

🎯 Key Takeaways for the Interview

- ConcurrentHashMap inserts into empty bins via lock-free CAS operations.
- Only the root node of the target hash bin is synchronized during collisions.
- ConcurrentHashMap strictly forbids null keys and values to prevent race ambiguities.

32

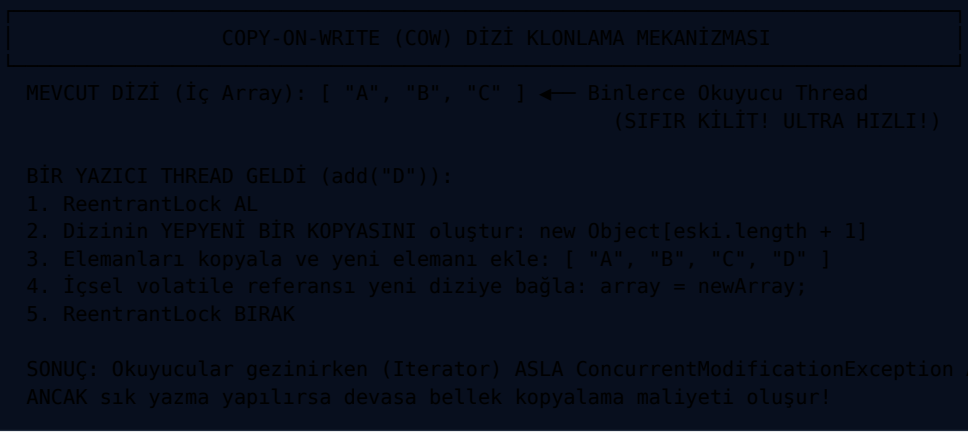
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

CopyOnWriteArrayList: Anlık Görüntü (Snapshot) İterasyonu & Yazma Cezası

CopyOnWriteArrayList: Snapshot Iteration, Write Penalties & When to Use It

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketin telefon rehberini düşünün. 1000 çalışan gün boyunca rehbere bakıp numaraları arar (Okuyucular). Kimsenin birbirini beklemesine gerek yoktur. Ama şirkete yeni bir personel girdiğinde, sekreter mevcut rehberin üzerine karalama yapmaz! Masasındaki fotokopi makinesinden rehberin koca bir kopyasını çıkarır, yeni personeli o yeni kağıda yazar ve panodaki eski kağıdı yenisiyle değiştirir! Eski kağıdı okumakta olanlar rahatsız olmaz. Ama her saniye yeni personel giriyorsa, fotokopi makinesi yanar!

🇬🇧 ENGLISH (ELI5)

Think of a published corporate phone directory. Hundreds of employees browse it simultaneously without blocking (Readers). When an employee joins the company, the receptionist does not scribble on the active sheet. Instead, the receptionist photocopies the entire directory, appends the new name to the duplicate, and magnetically snaps the new copy onto the board! Readers actively holding the old sheet finish unperturbed. But if you have 10,000 updates a second, the photocopier catches fire!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CopyOnWrite Mimarisi ve Kullanım Alanları: 1) Snapshot İteratör Garantisi: `iterator()` çağrıldığı anda o anki dizinin referansını alır. Başka bir thread listeye 1 milyon eleman eklese veya silse bile, iteratör kendi anlık görüntüsünü (Snapshot) taramaya devam eder; asla `ConcurrentModificationException` fırlatmaz. 2) Ağır Yazma Cezası (O(N) Allocation): Her `add()`, `set()`, `remove()` işlemi dizinin tamamını `Arrays.copyOf` ile yeniden tahsis eder. 3) Ne Zaman Kullanılmalıdır?: 'Okuma Çok (%99+), Yazma Çok Nadir (%1)' olan sistemlerde: Event Listener listeleri, filtre zincirleri ve güvenlik kuralı tabloları için mükemmeldir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CopyOnWrite Architectural Economics: 1) Fail-Safe Snapshot Iteration: The iterator holds a reference to the exact array state at the moment `iterator()` was invoked. Mutations occurring during iteration swap the internal `volatile Object[] array` reference without modifying the iterator's snapshot array, eliminating `ConcurrentModificationException` and the need for locks during reads. 2) O(N) Write Amplification: Every write operation acquires an exclusive `ReentrantLock` and clones the underlying backing array (`Arrays.copyOf()`), inducing substantial GC allocation pressure. 3) Domain Fit: Exclusively viable for read-dominant collections such as observer listener lists, routing tables, and security filter chains.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Event Listener Havuzunda CopyOnWriteArrayList Kullanımı
import java.util.concurrent.CopyOnWriteArrayList;

public class SecurityAuditNotifier {
    // Listener listesi: Okuma çok sık, yeni listener ekleme çok nadirdir
    private final CopyOnWriteArrayList<String> securitySubscribers = new CopyOnWriteArrayList<>();

    public void subscribe(String serviceName) {
        securitySubscribers.add(serviceName); // Dizi kopyalanarak yazılır
    }

    public void broadcastSecurityAlert(String alertMessage) {
        // SIFIR KİLİT! İterasyon anında liste değişse bile patlamaz!
        for (String subscriber : securitySubscribers) {
            IO.println("Alert yollandı -> " + subscriber + ": " + alertMessage);
        }
    }

    void main() {
        var notifier = new SecurityAuditNotifier();
        notifier.subscribe("FraudService");
        notifier.subscribe("AuthGateway");
        notifier.broadcastSecurityAlert("CVE-2026-9999 tespit edildi!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı '100.000 elemanlı bir listeye sürekli veri ekleyeceğimiz bir sistemde CopyOnWriteArrayList kullanabilir miyiz?' diye sorar. Yanıt: 'KESİNLİKLE HAYIR! Her eleman eklemede 100.000 elemanlık dizi baştan kopyalanır; bellek ve işlemci tükenir, sistem OOM veya Garbage Collection duraklamalarıyla felç olur!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Performance Trap: 'Can we use CopyOnWriteArrayList for high-throughput stream ingestion?' Answer: 'ABSOLUTELY NOT! Appending 100,000 items creates 100,000 full array allocations and copies, consuming gigabytes of transient heap and collapsing GC pause times. Use ConcurrentLinkedQueue or Collections.synchronizedList instead.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CopyOnWriteArrayList okuma işlemlerinde sıfır kilit maliyetiyle çalışır.
- İteratörler anlık görüntü (snapshot) üzerinden gezinir; CME hatası imkansızdır.
- Yazma işlemleri diziyi baştan kopyaladığı için sık yazılan sistemlerde kullanılmaz.

🎯 Key Takeaways for the Interview

- CopyOnWriteArrayList provides lock-free, volatile-read performance for all queries.
- Iterators traverse an isolated array snapshot, guaranteeing immunity to CME.
- Writes incur full array re-allocation; strictly avoid in write-heavy environments.

33

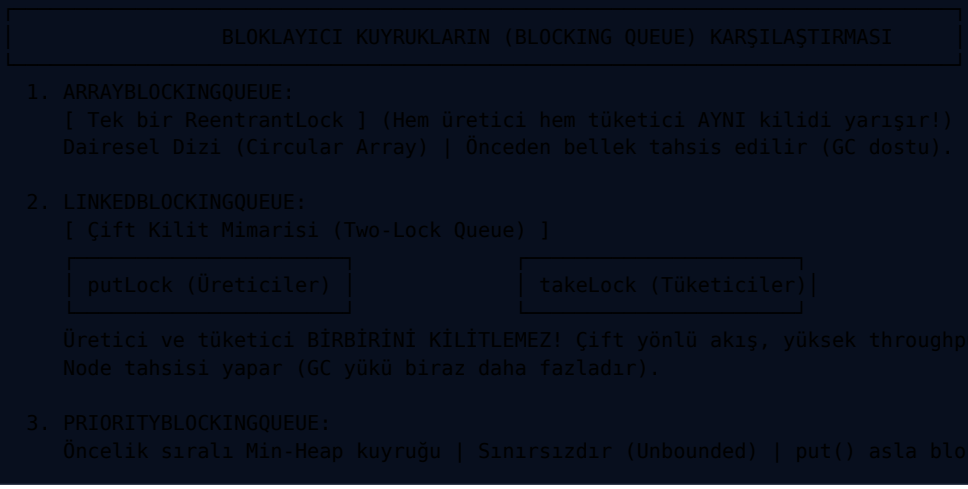
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Bloklayıcı Kuyruklar: ArrayBlockingQueue vs LinkedBlockingQueue vs PriorityBlockingQueue

Blocking Queues: ArrayBlockingQueue vs LinkedBlockingQueue vs PriorityBlockingQueue

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Fabrika çıkışındaki 3 farklı konveyör bandı gibidir: 1) ArrayBlockingQueue: Sabit 10 kutuluk metal bir tepsidir. Tepsi baştan ayrılmıştır, yeri sabittir. Ama tepsinin başında tek bir kilit vardır; aşçı yemek koyarken garson yemek alamaz. 2) LinkedBlockingQueue: Çift kapılı tüneldir. Tünelin girişinde ayrı kilit (putLock), çıkışında ayrı kilit (takeLock) vardır! Aşçı arkadan yemekleri yüklerken garson önden tepsileri alır; birbirlerinin ayağına basmazlar! 3) PriorityBlockingQueue: Acil servis kuyruğudur; durumu en acil olan hasta sıranın en önüne geçer!

🇬🇧 ENGLISH (ELI5)

Compare three factory conveyor belts: 1) ArrayBlockingQueue: A fixed 10-slot circular steel tray. Pre-allocated and bounded. However, it relies on a SINGLE lock shared between producers and consumers. 2) LinkedBlockingQueue: A two-door tunnel using 'Two-Lock Queue' topology. Producers contend for `putLock` at the entrance; consumers contend for `takeLock` at the exit. Producers and consumers operate concurrently without blocking each other! 3) PriorityBlockingQueue: A hospital triage queue where highest-priority elements jump to the head of the line via a binary heap.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

İki Kilitli Kuyruk Algoritması (Two-Lock Queue): 1) LinkedBlockingQueue'nun Başarımı: H.S. Chao ve Doug Lea'nın uyguladığı bu algoritmada, baş (head) ve kuyruk (tail) düğümleri iki ayrı `ReentrantLock` (`putLock` ve `takeLock`) ile korunur. Üreticiler sadece kuyruğun sonuna eleman eklerken, tüketiciler kuyruğun başından çeker. Aralarındaki tek bağ `AtomicInteger count` değişkenidir. Bu sayede eşzamanlı okuma ve yazma işlemlerinde `ArrayBlockingQueue`'dan 2-3 kat daha yüksek throughput üretir. 2) Bellek Farkı: ArrayBlockingQueue diziyi baştan tek parça tahsis eder, sıfır GC çöpü üretir. LinkedBlockingQueue her eleman için bir `Node` nesnesi tahsis eder; yüksek frekansta Young Generation GC baskısı yaratır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Two-Lock Queue Mechanics & Heap Trade-Offs: 1) LinkedBlockingQueue Concurrency Isolation: Employs a variation of the Michael & Scott Two-Lock Queue algorithm. Enqueue operations synchronize exclusively on `putLock` modifying `last`; dequeue operations synchronize exclusively on `takeLock` modifying `head`. Atomicity of capacity is preserved via `AtomicInteger count`. Producers and consumers rarely serialize against one another. 2) Garbage Collection Overhead: `ArrayBlockingQueue` allocates an `Object[]` array upfront, maintaining zero allocation churn at runtime. `LinkedBlockingQueue` instantiates a `Node\<E>` wrapper for every single item enqueued, generating GC churn in high-throughput allocations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: LinkedBlockingQueue ile Yüksek Hızlı Producer-Consumer Pipeline
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;

public class HighSpeedPipeline {
    void main() throws InterruptedException {
        // ⚠ Üretimde mutlaka bounded kapasite verilmelidir!
        var queue = new LinkedBlockingQueue<String>(10_000);

        // Üretici (Producer) - putLock üzerinde çalışır
        Thread.ofPlatform().name("IngestionWorker").start(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    queue.put("Mesaj-" + i);
                    IO.println("Kuyruğa eklendi: Mesaj-" + i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Tüketici (Consumer) - takeLock üzerinde çalışır (Çakışma YOK!)
        Thread.ofPlatform().name("ExportWorker").start(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    String msg = queue.poll(1, TimeUnit.SECONDS);
                    IO.println("Kuyruktan işlendi: " + msg);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'ArrayBlockingQueue mu yoksa LinkedBlockingQueue mu daha az bellek tüketir ve GC dostudur?' diye sorar. Yanıt: 'ArrayBlockingQueue çok daha GC dostudur çünkü baştan tek bir dizi tahsis eder, çalışma anında hiç yeni nesne (Node) üretmez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Interview Nuance: 'Which queue exhibits superior GC profile under extreme load?' Senior Answer: 'ArrayBlockingQueue! It allocates its backing array once during construction, generating zero runtime node allocations. LinkedBlockingQueue allocates a Node instance for every put, inducing significant allocation churn.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ArrayBlockingQueue tek kilit kullanır; LinkedBlockingQueue çift kilit (put/take) kullanır.
- LinkedBlockingQueue üretici ve tüketiciyi ayırdığı için daha yüksek throughput sunar.
- ArrayBlockingQueue baştan dizi tahsis ettiği için sıfır GC çöpü üretir.

🎯 Key Takeaways for the Interview

- ArrayBlockingQueue uses a single lock; LinkedBlockingQueue uses independent put/take locks.
- Dual locks grant LinkedBlockingQueue higher concurrency by decoupling producers from consumers.
- ArrayBlockingQueue avoids GC churn by reusing its preallocated array storage.

34

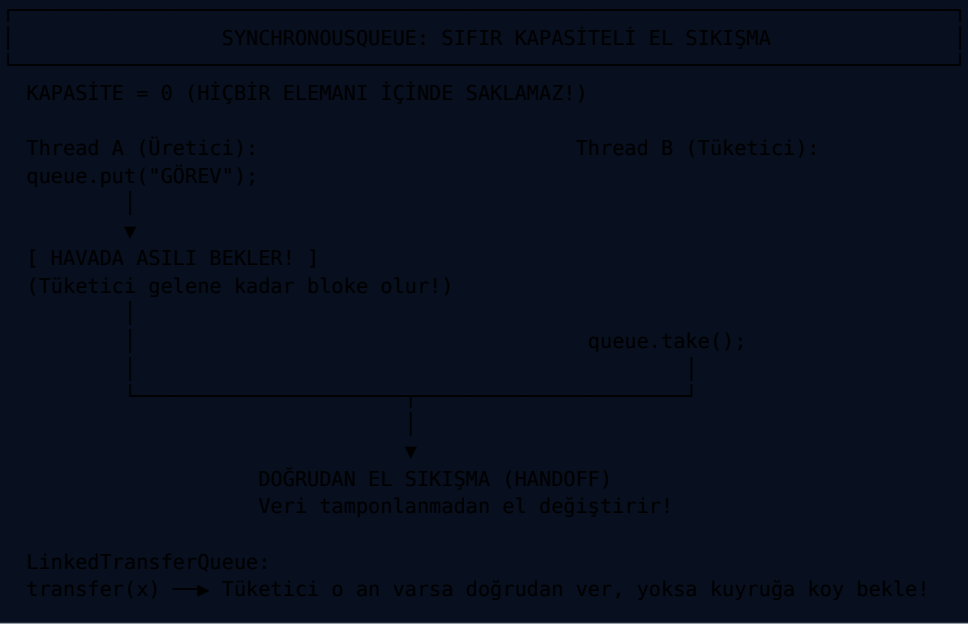
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Sıfır Kapasiteli El Sıkışma: SynchronousQueue vs TransferQueue

Zero-Capacity Direct Handoffs: SynchronousQueue vs TransferQueue

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Normal bir restoranda aşçı yemeği yapar, tezgahın üzerine koyar (Tamponlu kuyruk). SynchronousQueue'da ise tezgah YOKTUR (Kapasite = 0)! Aşçı sıcak tabağı elinde havada tutar (`put()`). Bir garson gelip tabağı aşçının elinden alana kadar (`take()`) aşçı kaskatı bekler! Tabak elden ele doğrudan teslim edilir. Arada hiçbir bekleme masası yoktur. Eğer hemen teslim alamayacaksan, aşçı yemeği pişirmeye bile kalkışmaz!

🇬🇧 ENGLISH (ELI5)

In a standard cafeteria, chefs set plates onto a warming counter (Buffering). A SynchronousQueue HAS NO COUNTER (Capacity = 0). The chef holds the scalding plate in mid-air (`put()`). The chef stands frozen in place until a server arrives to take the plate directly from their hands (`take()`). The transfer is an immediate atomic handoff from one thread's registers to another. If no server is waiting, the producer blocks instantly.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Doğrudan El Sıkışma (Direct Handoff) ve Dual Stack/Queue Mimarisi: 1) Sıfır Kapasite Felsefesi: `SynchronousQueue.size()` daima 0 döner; `peek()` daima null döner. Bir elemanı kuyruğa koymak ile o elemanın başka bir thread tarafından teslim alınması aynı zaman diliminde gerçekleşmek zorundadır. 2) Non-blocking Dual Queue (Scherer-Scott Algoritması): SynchronousQueue içsel olarak kilit kullanmaz; bekleyen üreticileri veya tüketicileri temsil eden düğümleri (QNode) lock-free bir yığında (TransferStack) veya kuyrukta (TransferQueue) CAS ile yönetir. 3) LinkedTransferQueue: Java 7 ile gelen bu yapı hem `BlockingQueue` hem de `TransferQueue` özelliklerini birleştirir. `tryTransfer(e)` metodu eğer bekleyen tüketici varsa anında el sıkışır, yoksa beklemeden false döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Zero-Capacity Handoff Mechanics: 1) The Non-Buffering Invariant: `SynchronousQueue` has zero internal storage. `put(e)` blocks until a concurrent `take()` arrives; `take()` blocks until a concurrent `put(e)` arrives. 2) Scherer-Scott Dual Stack/Queue: Implemented via lock-free dual data structures. Nodes represent either data requests or data fulfillment. Threads match against opposite-mode nodes using atomic CAS operations, eliminating monitor overhead. 3) `LinkedTransferQueue`: Blends `LinkedBlockingQueue` buffering with `SynchronousQueue` handoffs. The `transfer()` method blocks until a consumer accepts the item; `tryTransfer()` offers instant zero-wait handoffs.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: SynchronousQueue ile Doğrudan Handoff Gösterimi
import java.util.concurrent.SynchronousQueue;

public class DirectHandoffDemo {
    void main() throws InterruptedException {
        var syncQueue = new SynchronousQueue<String>();

        // Tüketici Thread (1 saniye geç başlıyor)
        Thread.ofPlatform().name("Consumer").start(() -> {
            try {
                Thread.sleep(1000);
                IO.println("Tüketici geldi, veriyi elden alıyor...");
                String item = syncQueue.take();
                IO.println("Tüketici teslim aldı: " + item);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Üretici Thread
        IO.println("Üretici: Veri teslim edilmeye çalışılıyor (put)...");
        long start = System.currentTimeMillis();
        syncQueue.put("Ultra-Önemli VIP Mesaj"); // Tüketici gelene kadar 1 saniye burada BLOKE OLUR!
        long elapsed = System.currentTimeMillis() - start;

        IO.println("Üretici: Veri el sıkışarak teslim edildi! Geçen süre: " + elapsed + " ms");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Executors.newCachedThreadPool() hangi kuyruğu kullanır ve neden?' Yanıt: 'SynchronousQueue kullanır! Çünkü gelen görevi hiçbir ara tamponda bekletmeden hemen boşta bekleyen bir thread'e teslim etmek ister; boşta thread yoksa anında yeni bir thread açar!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Crucial Interview Question: 'Which queue powers `Executors.newCachedThreadPool()` and why?' Answer: 'It uses `SynchronousQueue`! It demands immediate direct handoff to an idle worker. If all workers are busy, the handoff fails immediately, forcing the pool to spawn a new native thread instantly.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- SynchronousQueue'nun kapasitesi sıfırdır; veriyi asla bellekte tutmaz.
- put() çağrısı başka bir thread take() çağırana kadar bloke bekler.
- Executors.newCachedThreadPool doğrudan handoff için SynchronousQueue kullanır.

🎯 Key Takeaways for the Interview

- SynchronousQueue has zero internal capacity and never buffers elements.
- put() blocks until a matching take() arrives, executing an atomic thread-to-thread handoff.
- newCachedThreadPool relies on SynchronousQueue to enforce immediate task assignment.

35

BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Kilit-Siz Kuyruklar: ConcurrentLinkedQueue & Michael-Scott Algoritması

Lock-Free Queues: ConcurrentLinkedQueue & The Michael-Scott Algorithm

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              MICHAEL-SCOTT LOCK-FREE BAĞLI LİSTE KUYRUĞU │
└──────────────────────────────────────────────────────────┘

[ DUMMY HEAD NODE ] ──▶ [ NODE 1 ] ──▶ [ NODE 2 ] ──▶ null
        ▲                                   ▲
        |                                   |
   head pointer                        tail pointer

YENİ ELEMAN EKLEME (ENQUEUE - 2 ADIMLI CAS):
Adım 1: tail.next referansını CAS ile yeni düğüme bağla:
        [ NODE 2 ].next = [ YENİ NODE 3 ] ──▶ (CAS Başarılı!)

Adım 2: tail pointer'ını bir ileri kaydır:
        tail = [ YENİ NODE 3 ] ──▶ (CAS ile kaydır)

YARDIMLAŞMA PRENSİBİ (HELPING):
Eğer bir thread Adım 1'i yapıp Adım 2'yi yapamadan CPU'dan düşerse;
Gelen başka bir thread tail'in geride kaldığını fark eder ve tail'i ONUN YERİNE
ileri kaydırır (Helping mechanism)! Sistem ASLA kilitlenmez!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bayrak yarışı düşünün. Koşucular bayrağı birbirine devrederken sırayı durdurup hakemden izin almazlar (Kilit yok). Eğer bir koşucu bayrağı bir sonraki arkadaşına uzatıp tam çizgiye basamadan ayağı takılıp düşerse, arkadan gelen takım arkadaşı onun düşmesine aldırmaz; hemen eğilip onun yapamadığı adımı onun yerine tamamlar (Helping)! Kimse durmaz, bayrak yarışı ışık hızında devam eder!

🇬🇧 ENGLISH (ELI5)

Imagine a relay race with no referee stopping the clock for handoffs (Lock-Free). If Runner A clips the baton to Runner B's hand but trips before advancing the lane marker, Runner C arriving behind them doesn't panic. Runner C notices the unadvanced marker, nudges it forward on Runner A's behalf (The Helping Mechanism), and proceeds! Nobody waits for fallen runners; active threads complete stalled work cooperatively!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Michael-Scott Kuyruk Algoritması Derinliği: 1) Non-Blocking Garanti: Maged M. Michael ve Michael L. Scott tarafından 1996'da tasarlanan bu algoritma, `ConcurrentLinkedQueue`'nun kalbidir. Sistemin bütününde en az bir thread daima ilerleme (Progress guarantee) kaydeder. 2) Dummy Node (Yalancı Baş Düğüm): Kuyruk hiçbir zaman tamamen boş kalmaz; daima verisiz bir dummy node ile başlar. Bu sayede `head == tail` durumu özel bir kontrol gerektirmez. 3) Hop Count Optimizasyonu: HotSpot uygulamasında tail her `offer()` çağrısında değil, iki adımda bir güncellenir (`hops >= 2`). Bu sayede her eklemede iki kez CAS yapma maliyeti yarıya indirilir! 4) Bloklanmaz (Non-Blocking): `take()` gibi bloklayan bir metodu yoktur; boşsa `poll()` anında `null` döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Michael-Scott Lock-Free Queue Mechanics: 1) Theoretical Significance: Formally proven non-blocking queue published in 1996, foundational to `ConcurrentLinkedQueue`. It guarantees lock-free system-wide progress. 2) The Sentinel Dummy Node: The queue is initialized with a dummy sentinel node where `head == tail`. Enqueue operations never encounter an empty reference, eliminating null checks during concurrent pointer updates. 3) The Helping Hand Protocol: Enqueue is a two-step operation: swing `tail.next` to the new node, then swing `tail` to the new node. If Thread A stalls after step 1, Thread B detects `tail.next != null` and autonomously completes step 2 for Thread A before proceeding with its own enqueue! 4) Hop Batching: HotSpot batches tail updates every two hops, halving the number of expensive CAS cache-invalidation cycles.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ConcurrentLinkedQueue ile Kilit-Siz Yüksek Hızlı Kuyruk
import java.util.concurrent.ConcurrentLinkedQueue;

public class LockFreeQueueBenchmark {
    void main() throws InterruptedException {
        var queue = new ConcurrentLinkedQueue<Integer>();

        int producers = 4;
        int itemsPerProducer = 100_000;
        Thread[] threads = new Thread[producers];

        long start = System.currentTimeMillis();

        for (int i = 0; i < producers; i++) {
            threads[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < itemsPerProducer; j++) {
                    queue.offer(j); // SIFIR KİLİT! Tamamen CAS ile ekleme
                }
            });
        }

        for (Thread t : threads) t.join();
        long elapsed = System.currentTimeMillis() - start;

        IO.println("400.000 eleman kilit-siz eklendi. Süre: " + elapsed + " ms");
        IO.println("Kuyruk boyutu: " + queue.size());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Performans Uyarısı: `ConcurrentLinkedQueue.size()` metodunu döngü içinde KESİNLİKLE çağırmayın! Çünkü kilit-siz kuyrukta sabit bir `size` değişkeni tutulamaz (tüm thread'lerin o değişkende kilitlenmesi gerekirdi). Bu yüzden `size()` metodu her çağrıldığında tüm kuyruğu baştan sona tek tek sayar (O(N) karmaşıklık)! Kuyruğun boş olup olmadığını anlamak için `isEmpty()` kullanılmalıdır (O(1)).

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Severe Performance Trap: Never invoke `ConcurrentLinkedQueue.size()` in a tight loop! Because a lock-free queue cannot maintain an atomic size counter without crippling CAS scalability, `size()` iterates the entire linked list from head to tail ($O(N)$ runtime). Always invoke `isEmpty()` which checks `head.next == null` in $O(1)$ time.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentLinkedQueue, Michael-Scott kilit-siz kuyruk algoritmasını kullanır.
- Helping mekanizması sayesinde geri kalan pointer'lar diğer thread'ler tarafından tamamlanır.
- size() metodu O(N) karmaşıklıktadır; boşluk kontrolü için daima isEmpty() kullanılmalıdır.

🎯 Key Takeaways for the Interview

- ConcurrentLinkedQueue is built upon the Michael-Scott non-blocking queue algorithm.
- Cooperative helping allows active threads to swing stalled tail pointers forward.
- size() runs in O(N) linear time; always use isEmpty() for O(1) empty checks.

36

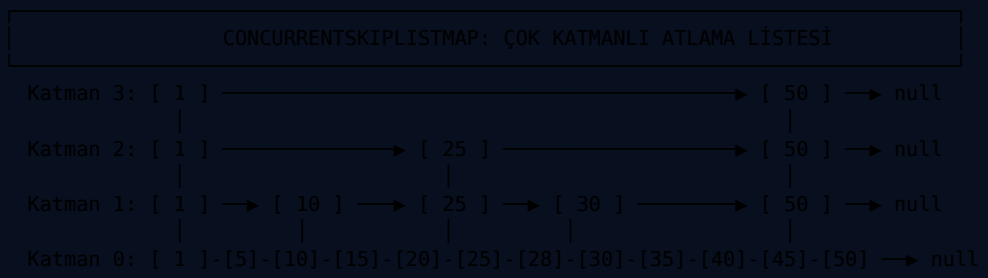
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

ConcurrentSkipListMap: Thread-Safe Sıralı Haritalar & Logaritmik Arama

ConcurrentSkipListMap: Thread-Safe Sorted Maps & Logarithmic Skip-Lists

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tren hattı düşünün. En alttaki tren her istasyonda durur (Yerel banliyö treni: 1, 2, 3, 4…). Yolculuk saatler sürer. Ama üstte bir ekspres tren vardır; sadece büyük şehirlerde durur (1, 25, 50…). Siz önce ekspres trene binip hedefinize en yakın büyük şehre gidersiniz, oradan inip yerel trene binersiniz! SkipList tam olarak budur: Sıralı veriyi kilit kullanmadan O(log N) hızında arayan çok katmanlı ekspres tren hattıdır!

🇬🇧 ENGLISH (ELI5)

Think of an express subway system. The local train on the bottom track stops at every single station (1, 2, 3… 50). Riding it takes forever. But directly above it runs an Express line stopping only at major hubs (1, 25, 50). To find Station 30, you ride the top express line to Station 25, hop down one flight of stairs to the local track, and arrive at 30 in seconds! SkipLists provide lock-free logarithmic search (O(log N)) without rebalancing locks!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ConcurrentSkipListMap vs TreeMap Mimarisi: 1) TreeMap Neden Thread-Safe Yapılamaz?: Kırmızı-Siyah Ağaçlar (Red-Black Trees) ekleme veya silme yapıldığında ağacı dengelemek (Rotation) zorundadır. Ağaç döndürme işlemi kök düğüm dahil birden fazla düğümü kilitler; kilit-siz (lock-free) bir AVL veya Red-Black Tree yazmak pratikte imkansızdır. 2) William Pugh Skip-List Mimarisi: Düğümler olasılıksal olarak yazıtura atılarak (Coin Flipping) rastgele katmanlara terfi ettirilir. Ekleme ve silme sadece yerel komşu düğümler üzerinde CAS ile yapılır; global ağaç döndürme gerekmez! 3) Thread-Safe Sıralı Gezinme: `subMap()`, `headMap()`, `tailMap()` gibi aralık (Range-Query) sorguları çoklu thread ortamında yüksek başarımla çalışır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

SkipList Topology vs Red-Black Trees: 1) The Impossibility of Lock-Free Trees: Balancing binary search trees (AVL, Red-Black) requires multi-node structural rotations. Synchronizing rotations across concurrent mutations creates massive contention bottlenecks, making lock-free balanced trees practically intractable. 2) Probabilistic Level Promotion: William Pugh's Skip-List utilizes random coin flips (1/2 probability) to build index levels. Mutations require CAS modifications strictly on local adjacent pointers without structural rebalancing! 3) Range Query Superiority: `ConcurrentSkipListMap` delivers thread-safe NavigableMap semantics, enabling concurrent subMap and range scans with bounded O(log N) search latency under heavy concurrent writes.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ConcurrentSkipListMap ile Sıralı Liderlik Tablosu (Leaderboard)
import java.util.concurrent.ConcurrentSkipListMap;

public class HighFrequencyLeaderboard {
    // Skorlara göre doğal sıralı thread-safe harita
    private final ConcurrentSkipListMap<Integer, String> scores = new ConcurrentSkipListMap<>();

    public void addScore(int score, String player) {
        scores.put(score, player); // O(log N) kilit-siz ekleme
    }

    public void printTopScores(int minScore) {
        // minScore üzerindeki oyuncuları sıralı getir (Range Query)
        var topTier = scores.tailMap(minScore);
        IO.println("Liderlik Tablosu (" + minScore + "+ Puan):");
        topTier.descendingMap().forEach((score, player) ->
            IO.println("• " + player + " : " + score + " Puan")
        );
    }

    void main() {
        var board = new HighFrequencyLeaderboard();
        board.addScore(9800, "Alice");
        board.addScore(4500, "Bob");
        board.addScore(12400, "Charlie");
        board.printTopScores(9000);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Sorusu: 'Collections.synchronizedSortedMap(new TreeMap()) varken neden ConcurrentSkipListMap kullanalım?' Yanıt: 'synchronizedSortedMap tek bir kaba kilit (Mutex) kullanır; aynı anda sadece tek bir thread işlem yapabilir. ConcurrentSkipListMap ise kilit-siz (lock-free CAS) çalışır; yüzlerce thread aynı anda haritayı okuyup yazabilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

MAANG Interview Comparison: 'Why prefer ConcurrentSkipListMap over Collections.synchronizedSortedMap(new TreeMap())?' Senior Answer: '`synchronizedSortedMap` wraps a single coarse-grained lock around every method, serializing all threads. `ConcurrentSkipListMap` employs lock-free CAS pointer manipulation on local nodes, scaling concurrently across hundreds of CPU cores.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentSkipListMap, TreeMap'in thread-safe ve kilit-siz alternatifidir.
- Skip-List yapısı rastgele katmanlarla O(log N) arama, ekleme ve silme sunar.
- Kırmızı-Siyah ağaç rotasyonlarına ihtiyaç duymadığı için lock-free ölçeklenir.

🎯 Key Takeaways for the Interview

- ConcurrentSkipListMap is the lock-free, thread-safe counterpart to TreeMap.
- Skip-Lists employ probabilistic index levels to achieve O(log N) operational bounds.
- Eliminating tree rotations allows skip-lists to scale concurrently under heavy writes.